

MAGIC STORY STUDIO

Documento de Mudanças Técnicas

Para implementação pela equipe Antigravity

Projeto	Magic Story Studio (MagicV3)
Data	27/05/2026
Stack	React + Vite + Supabase + Cloudflare Pages
Total de mudanças	7 mudanças (3 alta prioridade, 3 média, 1 baixa)

1. Resumo executivo

Com base no levantamento técnico respondido pela equipe Antigravity, foram identificados 7 pontos de melhoria no código. Os mais críticos envolvem o armazenamento de imagens e áudios base64 dentro do campo JSONB stories.data, o que é a principal causa do consumo excessivo de recursos no banco.

#	Mudança	Arquivo	Prioridade
1	Não salvar imagens base64 no JSONB — fazer upload para Storage	Studio/ImagesPage.tsx + storyStorage.ts	Alta
2	Não salvar áudio base64 no JSONB — fazer upload para Storage	Studio/NarrationPage.tsx + storyStorage.ts	Alta
3	Tratamento de erro 401 / token expirado no AuthContext	contexts/AuthContext.tsx	Alta
4	SELECT seletivo na listagem (sem carregar campo data)	lib/storyStorage.ts	Média
5	Paginação na listagem de histórias	lib/storyStorage.ts + Dashboard.tsx	Média
6	Completar variáveis no .env.example	.env.example	Média
7	Cache-Control longo nos uploads de thumbnail	lib/storyStorage.ts	Baixa

2. Problema raiz — base64 no JSONB

O principal problema identificado é que o campo `stories.data` armazena o `StudioState` completo, incluindo imagens e áudios como data URLs (base64). Uma única história com 5 cenas pode facilmente ter 10–30 MB nesse campo. Isso explica os 13 MB já alocados no banco mesmo com poucos registros.

Situação atual	Impacto	Solução
Imagem PNG base64 (~1.5 MB) dentro de <code>stories.data</code>	Banco cresce ~1.5 MB por cena gerada. Leitura lenta mesmo sem usar a imagem.	Upload para bucket thumbnails, salvar só a URL no state
Áudio WAV base64 dentro de <code>stories.data</code>	WAV não comprimido pode ter 3–10 MB por narração. Banco estoura rápido.	Upload para bucket story-audio (já existe), salvar só a URL
SELECT carrega campo data inteiro na listagem	Dashboard lento conforme histórias crescem.	SELECT seletivo sem o campo data na listagem

3. Mudanças detalhadas

Cada mudança abaixo inclui o arquivo exato, o código atual com o problema e o código corrigido. Implementar na ordem sugerida de prioridade.

3.1 Imagens — não salvar base64 no JSONB

Mudança 1 — <code>src/components/Studio/ImagesPage.tsx</code> + <code>src/lib/storyStorage.ts</code>	
Prioridade Alta	Fazer upload da imagem gerada para Storage antes de salvar no state Atualmente <code>scene.imageUrl</code> recebe a data URL completa (base64 PNG ~1.5 MB) e é persistida dentro de <code>stories.data</code> . Cada cena gerada infla o banco em ~1.5 MB. A correção é fazer upload para o bucket thumbnails imediatamente após gerar, e salvar apenas a URL pública no state.
✗ Código atual (problema) <pre>// src/components/Studio/ImagesPage.tsx // Atual - salva data URL diretamente no state const imageUrl = await generateImage(prompt); scene.imageUrl = imageUrl; // <-- base64 PNG vai para stories.data</pre>	
✓ Código corrigido <pre>// src/components/Studio/ImagesPage.tsx // Correto - faz upload e salva só a URL import { uploadSceneImage } from '../lib/storyStorage'; const base64DataUrl = await generateImage(prompt); const publicUrl = await uploadSceneImage(storyId, sceneIndex, base64DataUrl); scene.imageUrl = publicUrl; // <-- URL do Storage (~50 bytes)</pre>	

⚠ **Atenção:** Criar a função `uploadSceneImage` em `storyStorage.ts` conforme especificado abaixo.

Função a adicionar em `src/lib/storyStorage.ts`:

```
export async function uploadSceneImage(
  storyId: string,
  sceneIndex: number,
  dataUrl: string
): Promise<string> {
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error('Unauthenticated');

  // Converte data URL para Blob
  const res = await fetch(dataUrl);
  const blob = await res.blob();

  const filePath = `${user.id}/${storyId}_scene_${sceneIndex}.png`;
  const { error } = await supabase.storage
    .from('thumbnails')
    .upload(filePath, blob, {
      upsert: true,
      cacheControl: '604800', // 7 dias
      contentType: 'image/png',
    });
  if (error) throw error;

  const { data } = supabase.storage
    .from('thumbnails')
    .getPublicUrl(filePath);
  return data.publicUrl;
}
```

3.2 Áudio — não salvar base64 no JSONB

Mudança 2 — `src/components/Studio/NarrationPage.tsx` + `src/lib/storyStorage.ts`

Prioridade
Alta

Fazer upload do áudio WAV para Storage antes de salvar no state

O TTS retorna `data:audio/wav;base64,...` que é salvo em `audioUrl` dentro de `stories.data`. Um WAV de 30s tem ~3 MB. Com múltiplas narrações por história o banco estoura rapidamente. O bucket `story-audio` já existe (criado na migration 001). Basta fazer o upload antes de salvar no state.

✗ Código atual (problema)

```
// src/components/Studio/NarrationPage.tsx
// Atual — salva data URL de áudio no state
const audioPreviewUrl = await generateNarration(text);
scene.audioUrl = audioPreviewUrl; // <-- WAV base64 vai para stories.data
```

✓ Código corrigido

```
// src/components/Studio/NarrationPage.tsx
// Correto — faz upload e salva só a URL
```

```
import { uploadSceneAudio } from '../lib/storyStorage';

const audioDataUrl = await generateNarration(text);
const publicUrl = await uploadSceneAudio(storyId, sceneIndex,
audioDataUrl);
scene.audioUrl = publicUrl; // <-- URL do Storage
```

⚠ **Atenção:** O bucket story-audio já existe na migration 001_initial_schema.sql. Verificar se está criado no projeto Supabase — se não, rodar a migration.

Função a adicionar em src/lib/storyStorage.ts:

```
export async function uploadSceneAudio(
  storyId: string,
  sceneIndex: number,
  dataUrl: string
): Promise<string> {
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error('Unauthenticated');

  const res = await fetch(dataUrl);
  const blob = await res.blob();

  const filePath = `${user.id}/${storyId}_scene_${sceneIndex}.wav`;
  const { error } = await supabase.storage
    .from('story-audio')
    .upload(filePath, blob, {
      upsert: true,
      cacheControl: '604800',
      contentType: 'audio/wav',
    });
  if (error) throw error;

  const { data } = supabase.storage
    .from('story-audio')
    .getPublicUrl(filePath);
  return data.publicUrl;
}
```

3.3 AuthContext — tratamento de token expirado

Mudança 3 — src/contexts/AuthContext.tsx

Prioridade
Alta

Adicionar handling do evento TOKEN_REFRESHED_ERROR no onAuthStateChange

As políticas RLS foram migradas para {authenticated}. Se o token Supabase expirar durante o uso e não for renovado automaticamente, as queries retornam erro silencioso ou tela em branco. O onAuthStateChange precisa capturar SIGNED_OUT para redirecionar ao login.

✗ Código atual (problema)

```
// Atual - sem tratamento explícito de expiração
supabase.auth.onAuthStateChange((event, session) => {
```

```

    setUser(session?.user ?? null);
    setLoading(false);
  });

```

✓ Código corrigido

```

supabase.auth.onAuthStateChange((event, session) => {
  if (event === 'SIGNED_OUT' || event === 'TOKEN_REFRESHED') {
    if (!session) {
      setUser(null);
      // Redireciona para login se não houver sessão
      window.location.href = '/login';
      return;
    }
  }
  setUser(session?.user ?? null);
  setLoading(false);
});

// Adicionar também interceptor global de erro Supabase:
// Em qualquer service que faz query, tratar error.status === 401:
// if (error?.status === 401) { await supabase.auth.signOut(); }

```

3.4 SELECT seletivo na listagem de histórias

Mudança 4 — src/lib/storyStorage.ts

Prioridade
Média

Remover campo data do SELECT de listagem

A query de listagem em getAllStories() já faz SELECT seletivo correto — mas há risco de outros pontos carregarem o registro completo. Confirmar que nenhuma outra query usa .select('*') na tabela stories. O campo data (StudioState com base64) nunca deve ser carregado na listagem.

✗ Código atual (problema)

```

// Atual — query de listagem (já está correta, manter assim)
const { data, error } = await supabase
  .from('stories')
  .select('id, title, created_at, updated_at, preview_image,
    is_complete, currentStep:data->>currentStep')
  .order('updated_at', { ascending: false });

// Problema: StoryDashboard.tsx tem query separada SEM campo data,
// mas outros componentes podem chamar .select('*') acidentalmente.

```

✓ Código corrigido

```

// Garantir que NENHUMA query de listagem inclui o campo data.
// Criar constante com os campos permitidos na listagem:

// src/lib/storyStorage.ts — adicionar no topo
export const STORY_LIST_FIELDS =
  'id, title, created_at, updated_at, preview_image, is_complete';

// Usar em todos os SELECTs de listagem:
supabase.from('stories').select(STORY_LIST_FIELDS)

// Para carregar o state completo (dentro do Studio):
supabase.from('stories').select('id, data').eq('id', storyId)

```

⚠ **Atenção:** Buscar no projeto inteiro por .from('stories').select — verificar se algum usa '*' ou inclui o campo data em contexto de listagem.

3.5 Paginação na listagem

Mudança 5 — `src/lib/storyStorage.ts` +
`src/components/Dashboard.tsx`

Prioridade
Média

Adicionar paginação com limit/offset na query cloud

Atualmente todas as histórias do usuário são carregadas de uma vez. Com as mudanças 1 e 2 o campo data ficará menor, mas ainda assim o volume de registros pode crescer. Implementar paginação simples de 20 itens.

✗ Código atual (problema)

```
// Atual - sem paginação
const { data, error } = await supabase
  .from('stories')
  .select(STORY_LIST_FIELDS)
  .order('updated_at', { ascending: false });
// retorna TUDO
```

✓ Código corrigido

```
// Correto - com paginação
export async function getAllStories(page = 0, pageSize = 20) {
  const from = page * pageSize;
  const to = from + pageSize - 1;

  const { data, error, count } = await supabase
    .from('stories')
    .select(STORY_LIST_FIELDS, { count: 'exact' })
    .order('updated_at', { ascending: false })
    .range(from, to);

  if (error) throw error;
  return { stories: data, total: count, page, pageSize };
}

// No Dashboard.tsx: exibir botão 'Carregar mais' ou scroll infinito
```

3.6 Completar .env.example

Mudança 6 — `.env.example`

Prioridade
Média

Documentar todas as variáveis de ambiente usadas no projeto

O `.env.example` atual não lista `VITE_GOOGLE_CLOUD_API_KEY`, `VITE_VERTEX_AI_URL`, `GCP_PROJECT_ID`, `GCP_CREDENTIALS_JSON` e `GCP_REGION_TTS`. Isso dificulta o onboarding e pode causar erros silenciosos em deploys.

✗ Código atual (problema)

```
# Atual - incompleto
VITE_SUPABASE_URL=your_supabase_url_here
VITE_SUPABASE_ANON_KEY=your_supabase_anon_key_here
VITE_GEMINI_API_KEY=your_gemini_api_key_here
VITE_OPENAI_API_KEY=your_openai_api_key_here # Optional
```

✓ Código corrigido

```
# Completo
```

```
# Supabase
VITE_SUPABASE_URL=your_supabase_url_here
VITE_SUPABASE_ANON_KEY=your_supabase_anon_key_here

# Google AI
VITE_GEMINI_API_KEY=your_gemini_api_key_here
VITE_GOOGLE_CLOUD_API_KEY=your_google_cloud_api_key_here
VITE_VERTEX_AI_URL=your_vertex_ai_endpoint_here

# GCP (usadas nas Supabase Edge Functions / backend)
GCP_PROJECT_ID=your_gcp_project_id
GCP_CREDENTIALS_JSON='{...}' # JSON da service account
GCP_REGION_TTS=us-central1

# OpenAI (opcional)
VITE_OPENAI_API_KEY=your_openai_api_key_here
```

⚠ **Atenção:** As variáveis `GCP_*` são usadas nas Edge Functions do Supabase. Configurá-las também no painel: Supabase Dashboard → Edge Functions → Secrets.

3.7 Cache-Control no upload de thumbnail

Mudança 7 — `src/lib/storyStorage.ts`

Prioridade
Baixa

Aumentar Cache-Control de 1h para 7 dias nos thumbnails

Os thumbnails atuais têm `cacheControl: max-age=3600` (1 hora). Como são imagens estáticas geradas uma vez, podem ser cacheadas por 7 dias sem problemas. Isso reduz requisições repetidas ao Storage.

✗ **Código atual (problema)**

```
// Atual — cache de 1 hora (padrão Supabase)
supabase.storage
  .from('thumbnails')
  .upload(filePath, blob, { upsert: true });
// cacheControl não definido = 3600s (1h) por padrão
```

✓ **Código corrigido**

```
// Correto — cache de 7 dias
supabase.storage
  .from('thumbnails')
  .upload(filePath, blob, {
    upsert: true,
    cacheControl: '604800', // 7 dias em segundos
    contentType: 'image/png',
  });
```

4. Migração dos dados existentes

Após implementar as mudanças 1 e 2, as novas histórias salvarão URLs no state. As histórias antigas ainda têm base64 no campo `data`. O script abaixo deve ser executado uma vez para migrar os registros existentes.

Script a ser executado no Supabase SQL Editor ou como Edge Function:

```
-- Verificar quantas histórias têm base64 no campo data
SELECT id, title,
       pg_size_pretty(octet_length(data::text)) AS data_size
FROM stories
WHERE data::text LIKE '%data:image/png;base64%'
       OR data::text LIKE '%data:audio/wav;base64%'
ORDER BY octet_length(data::text) DESC;
```

A migração real (extração do base64, upload para Storage e substituição pela URL) deve ser feita via script Node.js ou Edge Function — disponibilizar se necessário após confirmação das mudanças acima.

5. Checklist de QA após implementação

	Teste	Resultado esperado
☐	Gerar imagem de cena e verificar em Supabase Storage → bucket thumbnails	Arquivo PNG criado
☐	Gerar narração e verificar em Supabase Storage → bucket story-audio	Arquivo WAV criado
☐	Verificar campo data no Supabase Table Editor — não deve conter 'data:image'	Sem base64
☐	Abrir Dashboard com 10+ histórias — verificar tempo de carregamento	< 1 segundo
☐	Expirar token manualmente (limpar cookie) e tentar navegar → deve redirecionar ao login	Redirect /login
☐	Verificar tamanho do banco no Supabase Dashboard → deve crescer pouco após gerar história	< 50 KB por história
☐	Thumbnails: inspecionar header de resposta do Storage — deve ter max-age=604800	Cache 7 dias

Documento gerado com base no levantamento técnico respondido pela equipe Antigravity em 27/05/2026